

IN2401 Database Management Systems

Introduction

Kanishka Nanayakkara
Lecturer
Department of Information Technology
University Of Moratuwa
kanishkan@uom.lk



Welcome to IN2401

#	Module Code IN2401
	Credits 3 credit hours
	Lecture Hours 2 hours per week
	Lab Work 2 hours per week
	Prerequisite IN1401 Fundamentals of Databases

Module Objectives



Theory Application

Apply database theories, models, and design principles to solve real-world problems



Enterprise Development

Design and develop scalable, enterprise-level database applications using industry standards



Administration Skills

Master database administration, including performance tuning, backup, and security management

Course Learning Outcomes

By the end of this module, you will be able to:

1 Design Database Systems

Create efficient database schemas using normalization and entity-relationship modeling

2 Write SQL Queries

Develop complex SQL statements for data manipulation and retrieval operations

3 Optimize Performance

Apply indexing, query optimization, and transaction management techniques

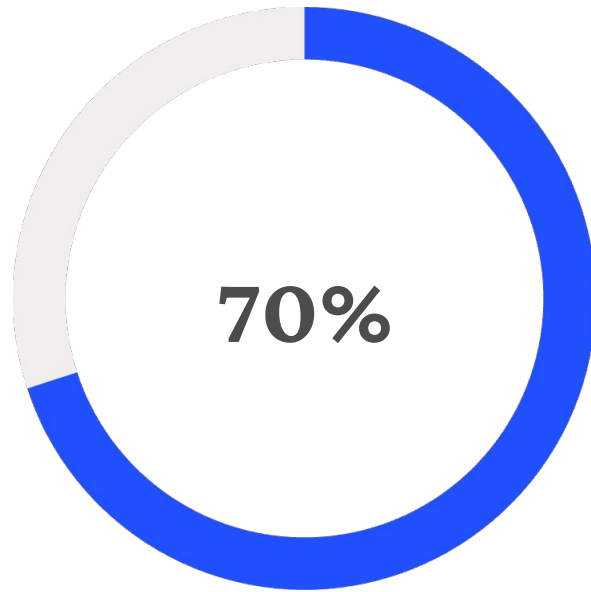
4 Ensure Data Security

Implement security measures including authentication, authorization, and encryption

5 Manage Databases

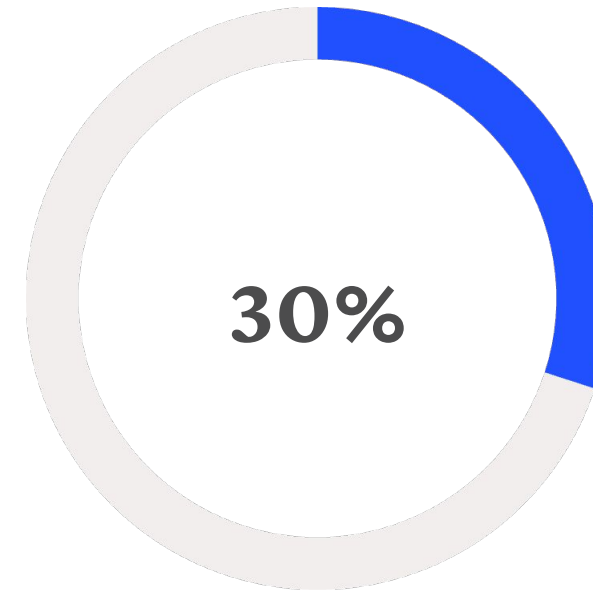
Handle backup, recovery, and maintenance procedures for production systems

Assessment Breakdown



Final Examination

written exam covering all module topics



Continuous Assessments

Assignments, quizzes, and practicals

Continuous Assessments

- Weekly lab Submissions
- Mid-semester quiz
- Practical exam



week 8 and week 12 (tentative)

Examination Eligibility Requirements



 **Note:** Please make sure you maintain good attendance and complete all assignments on time.



Revisiting What You Learned in IN1401

Database Fundamentals Recap for Database Management Systems

Database Fundamentals Recap

What is a Database?

An organized collection of structured data stored electronically, designed for efficient retrieval, updating, and management of information.

Popular DBMS Examples

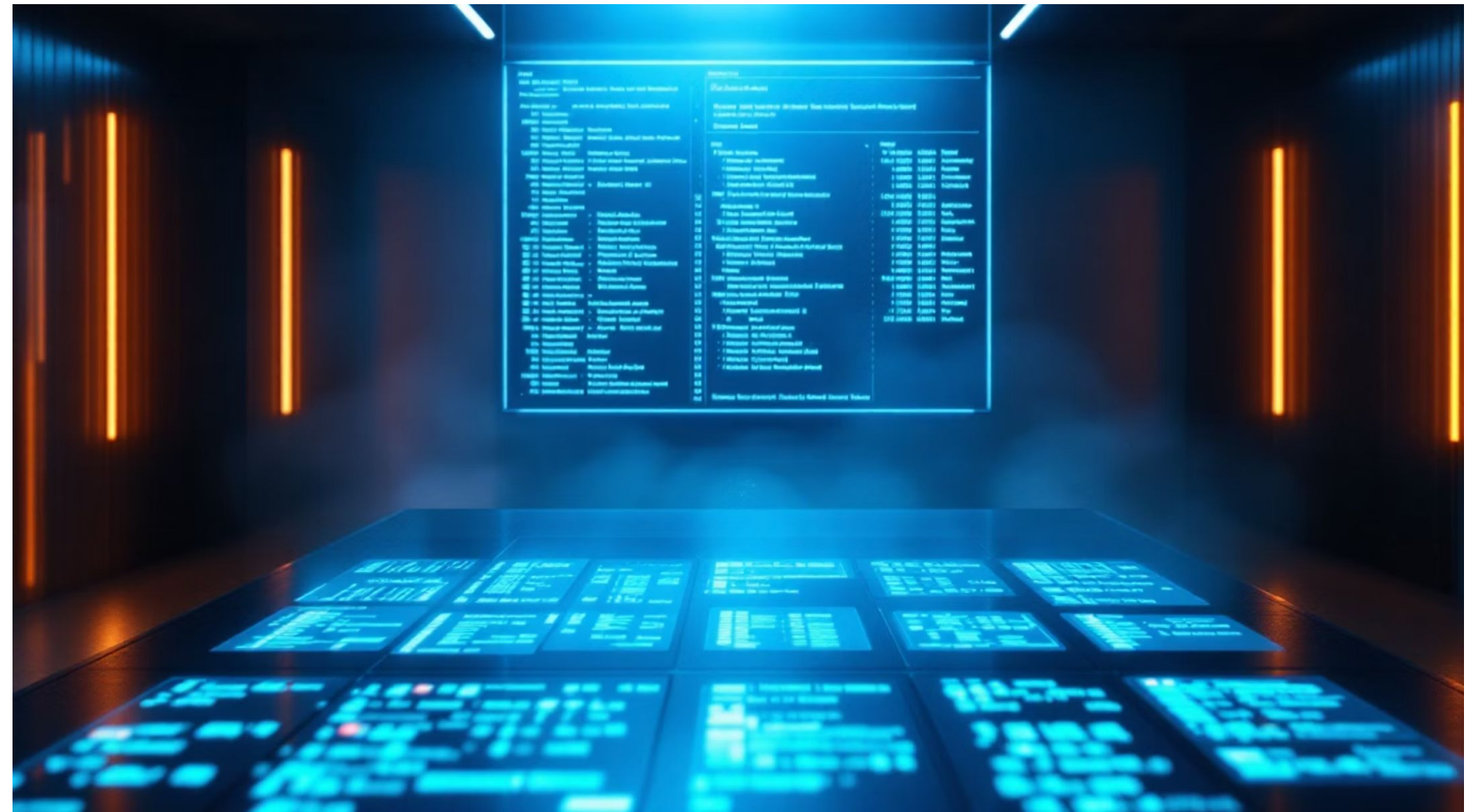
- MySQL
- PostgreSQL
- Oracle Database
- Microsoft SQL Server
- MongoDB

The Relational Database Model

Core Components

Relational databases organize data into structured tables that resemble spreadsheets. Each table represents a specific entity type, and the structure ensures data integrity and efficient querying.

- **Tables:** Store data about specific entities (e.g., Students, Courses)
- **Rows:** Represent individual records or entries
- **Columns:** Define attributes or properties of the data



Keys in Databases

Keys establish relationships between tables and ensure data integrity by uniquely identifying records.

1

Primary Key

Uniquely identifies each row in a table. Must contain unique values and cannot be NULL. Example: StudentID in a Students table.

2

Foreign Key

References the primary key of another table, establishing relationships between tables. Creates connections across the database structure.

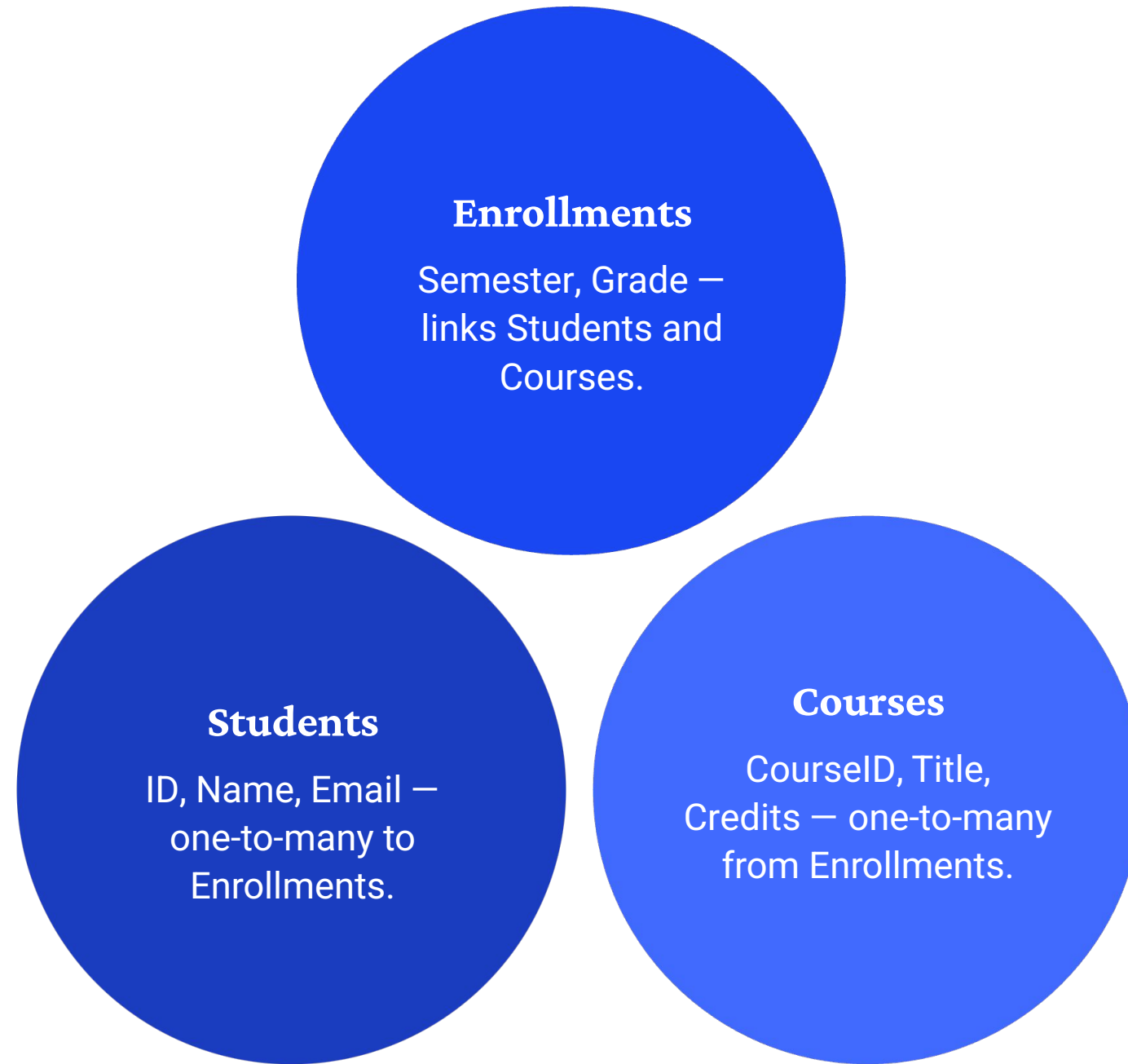
3

Candidate Key

Column or set of columns that could serve as a primary key. Contains unique values but hasn't been selected as the primary identifier.

📌 **Why Keys Matter:** Keys prevent duplicate data, maintain referential integrity, enable efficient joins between tables, and speed up data retrieval through indexing.

ER Model Recap



Ice Breaker Activity

Activity Scenario: University Library System

The officials decided that all information related to the airport should be organized using a DBMS, and you have been hired to design the database. Your first task is to organize the information about all the airplanes stationed and maintained at the airport. The relevant information is as follows:

- Every airplane has a registration number.
- Each airplane belongs to a specific model.
- The airport accommodates a number of airplane models. and each model is identified by a model number (e.g. DC-10) and has a capacity and a weight.
- A number of technicians work at the airport. You need to store the name, NIC, address, phone number, and salary of each technician. You can assume that each technician is uniquely identified by a national identity card (NIC) number.
- Each technician is an expert on one or more plane model(s)
- The airport has a number of test types, that are used periodically to ensure that airplanes are still airworthy. Each test type has Federal Aviation Administration (FAA) test number, a name, and a maximum possible score
- The FAA requires the airport to keep track of each time a given airplane is tested by a given technician using a given test. For each testing event, the information needed is the date and time, the number of hours the technician spent doing the test, and the score the airplane received on the test. Technicians are experts for doing the tests parallelly as well as for a given plane, several test types can be done parallel for a particular time.

Identify These Elements:

01

Entities

0

2
Attributes

0

3
Primary keys

Normalization Recap

Normalization eliminates redundant data and improves database structure by organizing tables efficiently.



1NF: First Normal Form

Eliminate repeating groups. Each column contains atomic (indivisible) values, and each row is unique.



2NF: Second Normal Form

Remove partial dependencies. All non-key attributes must depend on the entire primary key, not just part of it.



3NF: Third Normal Form

Remove transitive dependencies. Non-key attributes cannot depend on other non-key attributes—only on the primary key.



SQL Introduction

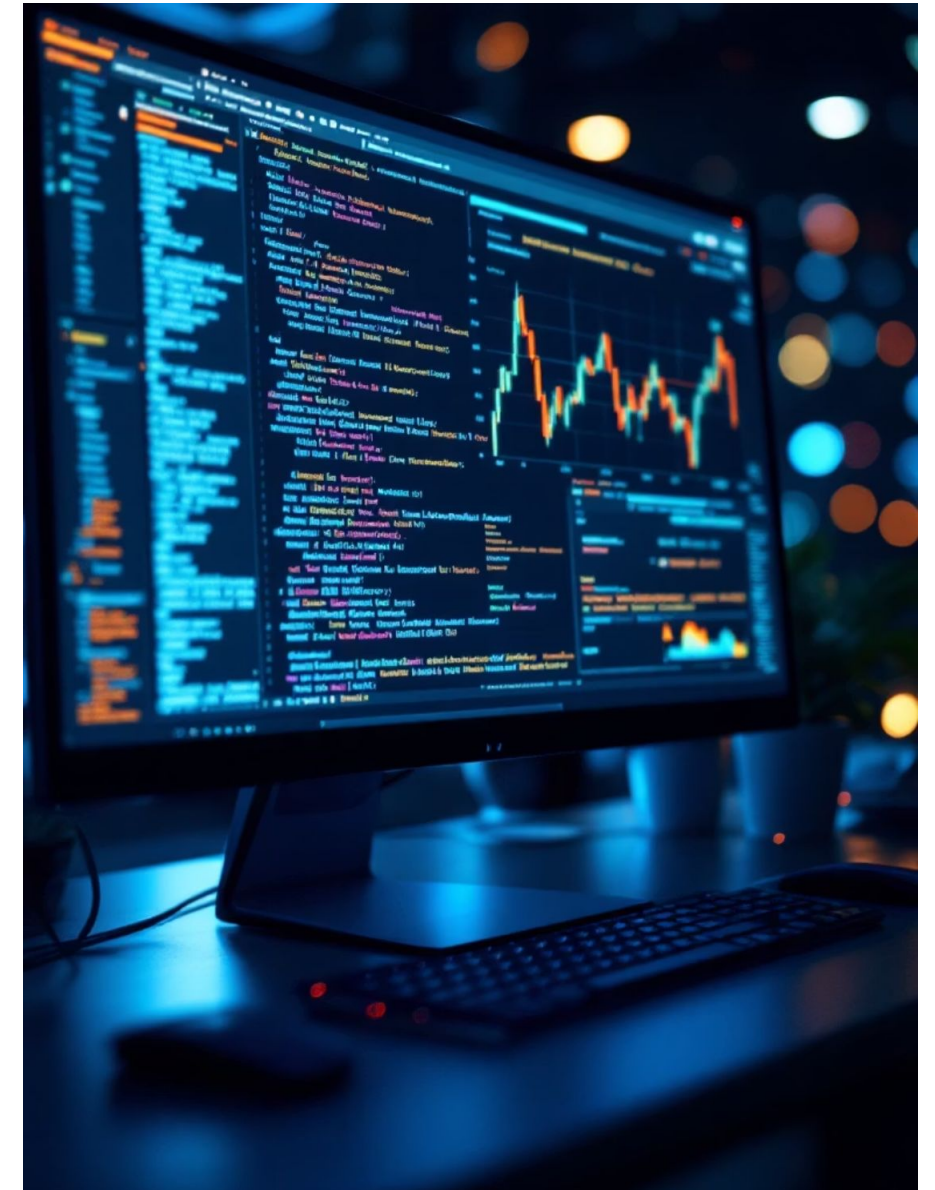
Recap

What is SQL?

Structured Query Language (SQL) is the standard programming language for managing relational databases. It provides a consistent way to communicate with database systems across different platforms.

Why SQL?

- Universal standard across all relational databases
- Declarative syntax makes queries intuitive
- Handles complex operations with simple commands
- Separates data from application logic
- Supports transactions and data integrity



SQL Basic Commands

These four commands form the foundation of data manipulation in SQL.



SELECT

Retrieves data from one or more tables. The most frequently used command for querying databases.

```
SELECT * FROM Students;
```



INSERT

Adds new rows of data into a table. Used for creating new records.

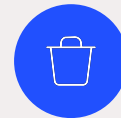
```
INSERT INTO Students VALUES (...);
```



UPDATE

Modifies existing data in a table. Changes values in specified rows.

```
UPDATE Students SET ... WHERE ...;
```



DELETE

Removes rows from a table. Careful: this permanently deletes data.

```
DELETE FROM Students WHERE ...;
```

SQL Example Queries

Working with a Students table containing StudentID, Name, Email, Major, and GPA fields.

Basic SELECT

```
SELECT Name, Major, GPA
FROM Students
WHERE GPA > 3.5;
```

Retrieves students with GPA above 3.5

INSERT Example

```
INSERT INTO Students
(StudentID, Name, Email, Major, GPA)
VALUES (101, 'John Doe',
'john@example.com', 'CS', 3.8);
```

Adds a new student record

UPDATE Example

```
UPDATE Students
SET GPA = 3.9
WHERE StudentID = 101;
```

Updates John Doe's GPA

DELETE Example

```
DELETE FROM Students
WHERE GPA < 2.0;
```

Removes students with GPA below 2.0

SQL Practice Activity

Given Table Structure

StudentID	Name	Major	GPA
101	Alice Smith	Biology	3.7
102	Bob Johnson	CS	3.9
103	Carol Davis	Math	3.5
104	David Wilson	CS	3.8

Your Task

Write a SQL query to select the **Name** and **Major** of all students whose **GPA** is greater than 3.6. sort results by GPA in descending order.

What is a Database Management System?



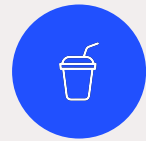
Data Storage

Organized, efficient storage of information using tables, documents, or graphs



Data Security

Protection through access controls, encryption, and backup mechanisms



Data Consistency

Maintaining accuracy and integrity across all operations and transactions

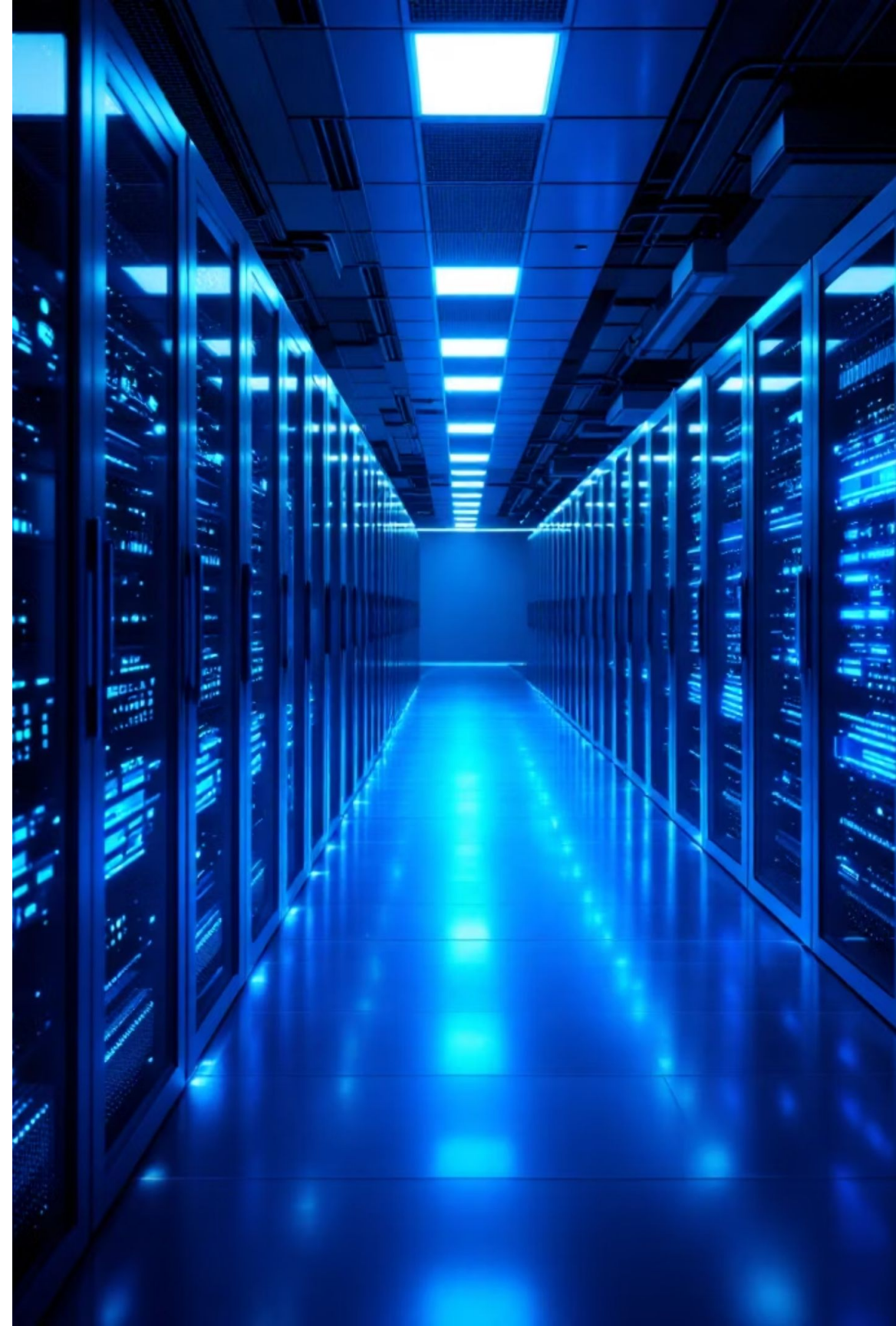


Multi-user Access

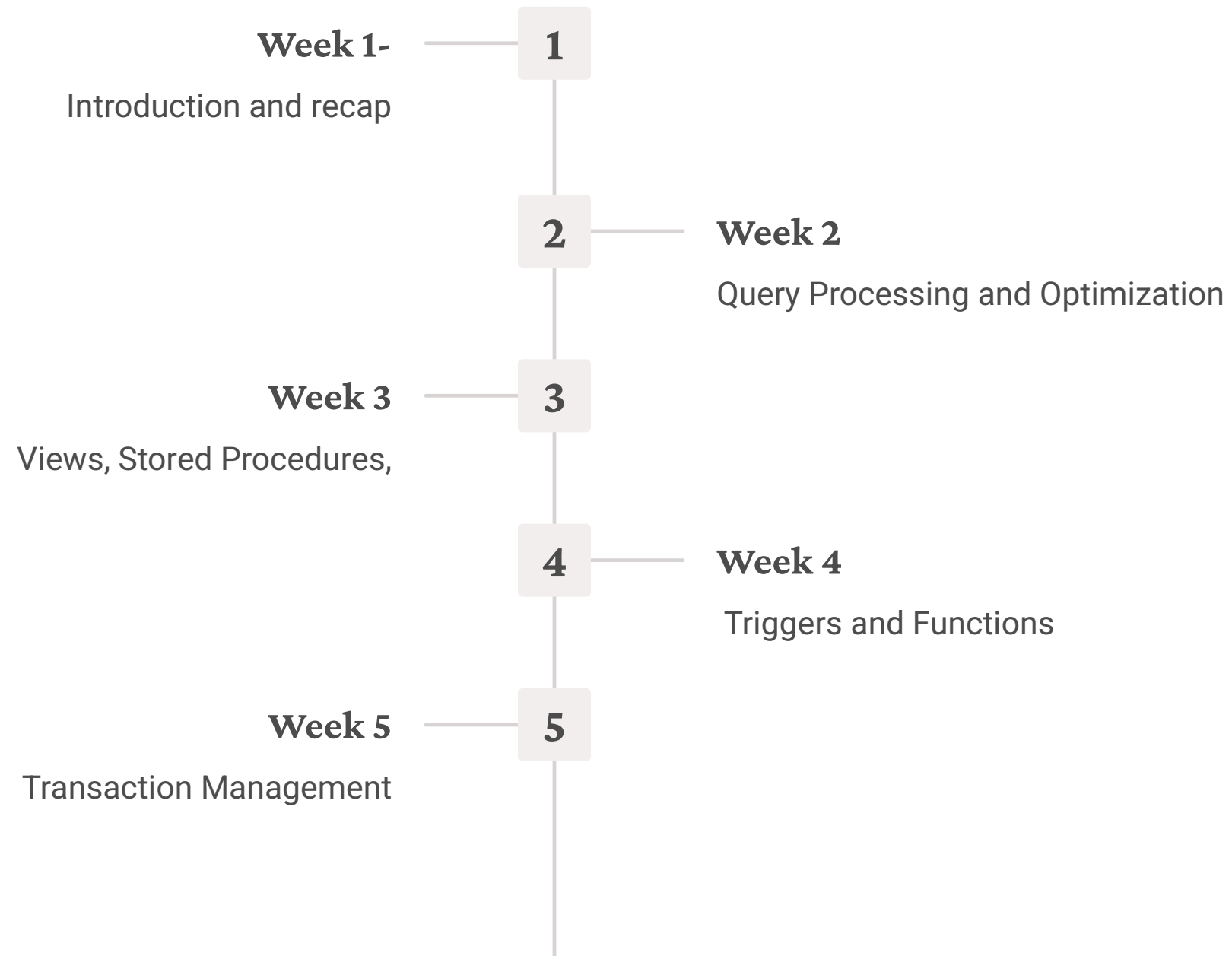
Concurrent access with controlled permissions and conflict resolution

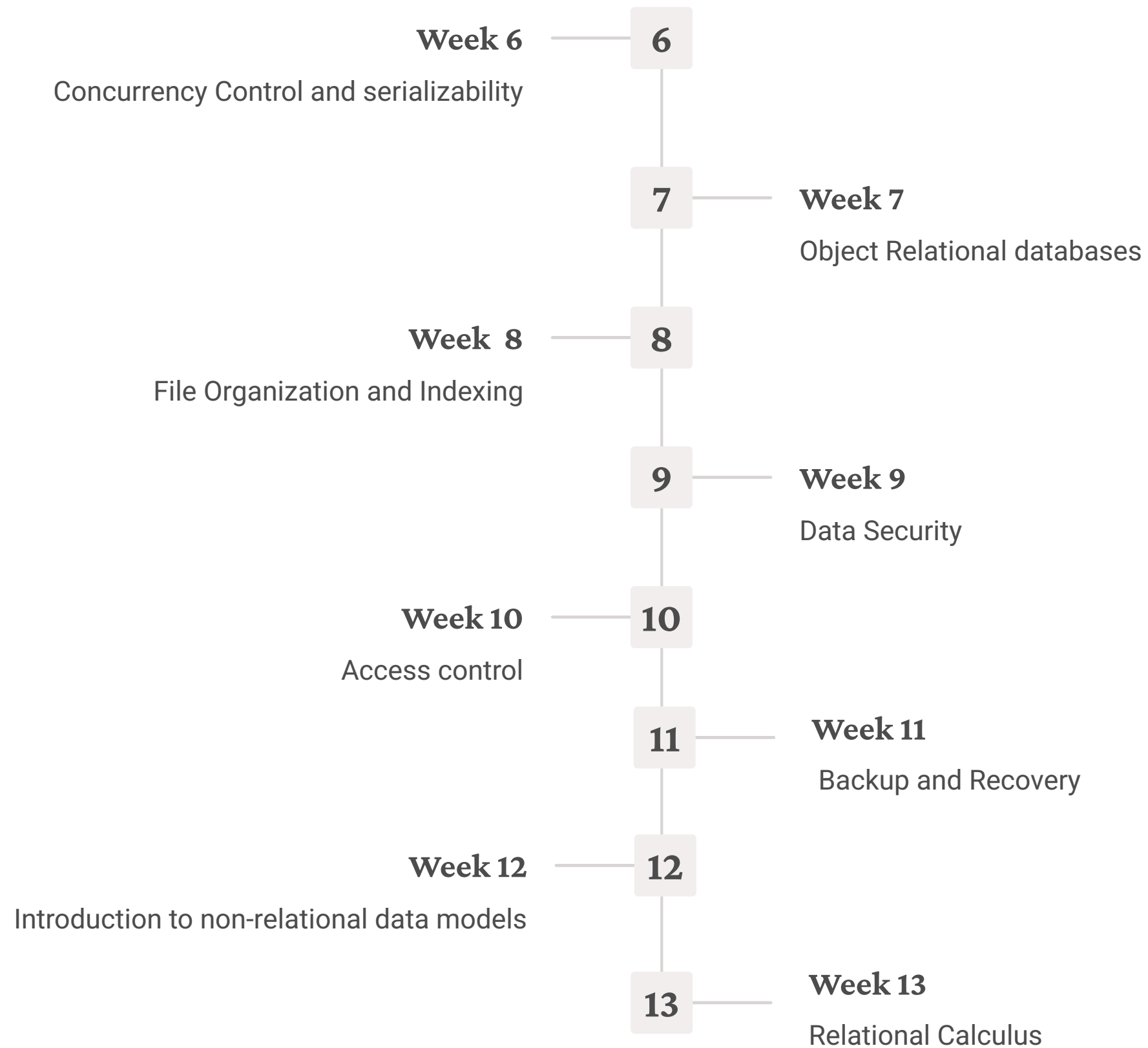
A DBMS serves as the foundation for virtually every modern application, managing how data is stored, retrieved, and protected in enterprise systems worldwide.

Topics We Will Explore in This Lecture Series



Lecture Series Overview





Query Processing and Optimization

01

Query Parsing

SQL statement is analyzed and validated for syntax and semantics

0

²Query Optimization

Multiple execution plans are evaluated and the most efficient one is selected

0

³Execution

Optimized plan is executed and results are returned to the user

Why optimization matters: Different query plans can have vastly different performance. A poorly optimized query might take minutes instead of milliseconds. The DBMS uses statistics and cost models to choose the fastest path.

Views, Stored Procedures, Triggers, and Functions

Views

Virtual tables that simplify complex queries and provide data abstraction. Users query a view just like a table, but the data is dynamically generated from underlying tables.

Stored Procedures

Precompiled collections of SQL statements stored in the database. They reduce network traffic and improve performance by executing logic server-side.

Triggers

Automatically execute in response to database events (INSERT, UPDATE, DELETE). Useful for enforcing business rules, auditing changes, and maintaining derived data.

Functions

Reusable code blocks that return values. Can be used in SQL statements to transform data or perform complex calculations.

Transaction Management

ACID Properties: The Foundation of Reliable Databases



Atomicity

All operations in a transaction succeed or fail together. No partial updates allowed.



Consistency

Database moves from one valid state to another, maintaining all integrity constraints.



Isolation

Concurrent transactions don't interfere with each other's intermediate states.



Durability

Once committed, changes survive system failures and power loss.



Concurrency Control and Serializability

The Challenge

Modern databases handle hundreds or thousands of users simultaneously. Without proper control, concurrent transactions could produce inconsistent results or corrupt data.

The Solution

Locking mechanisms and timestamp ordering ensure that concurrent execution produces the same results as some serial (one-at-a-time) execution order.



1

Locking

Prevents conflicting operations

2

Two-Phase Locking

Guarantees serializability

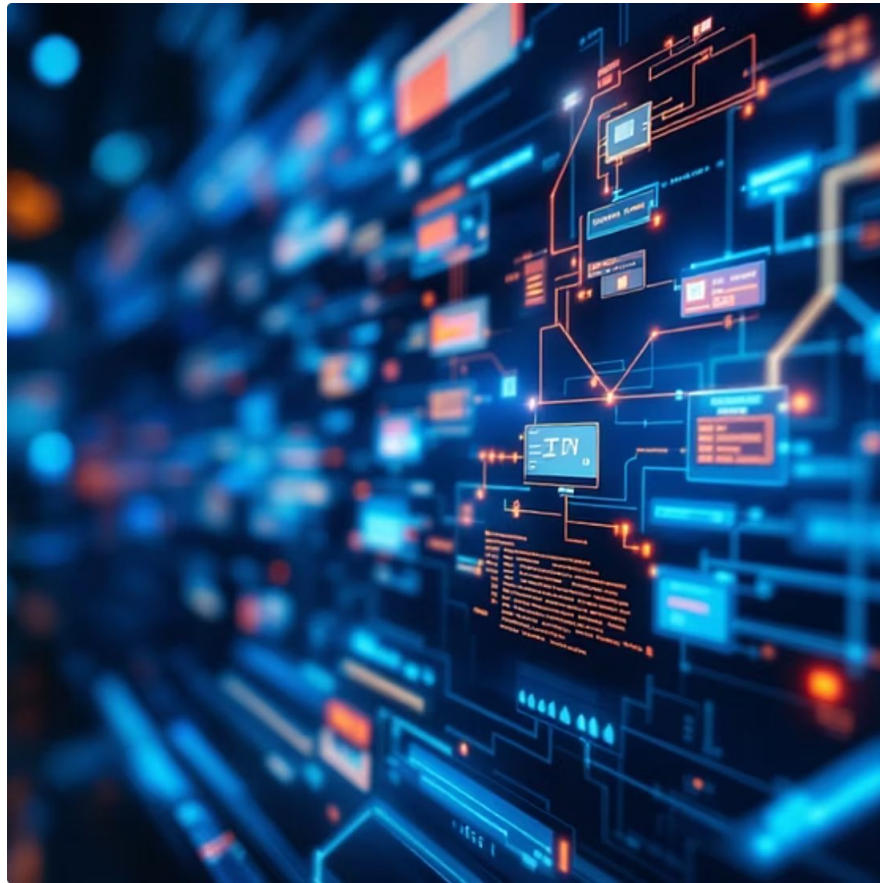
3

Deadlock Detection

Resolves circular waits

Object-Relational Databases

Object-relational databases combine the rigor of relational databases with the flexibility of object-oriented programming. They support user-defined data types, inheritance, and methods while maintaining SQL compatibility.



Inheritance

Tables can inherit attributes and methods from parent tables



Complex Types

Create custom data types like geometric shapes or multimedia



Methods

Attach functions to data types for encapsulated behavior

File Organization and Indexing

How Data Lives on Disk

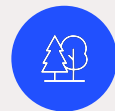
Databases store tables in files using various organizations:

- Heap files (unordered insertion)
- Sequential files (sorted by key)
- Hashed files (direct key access)
- Clustered files (related data together)



Indexes

Special data structures that provide rapid access to table rows without scanning entire tables



B-Trees

Most common index structure, optimized for disk storage with balanced tree depth



Query Speed

Proper indexing can reduce query time from minutes to milliseconds

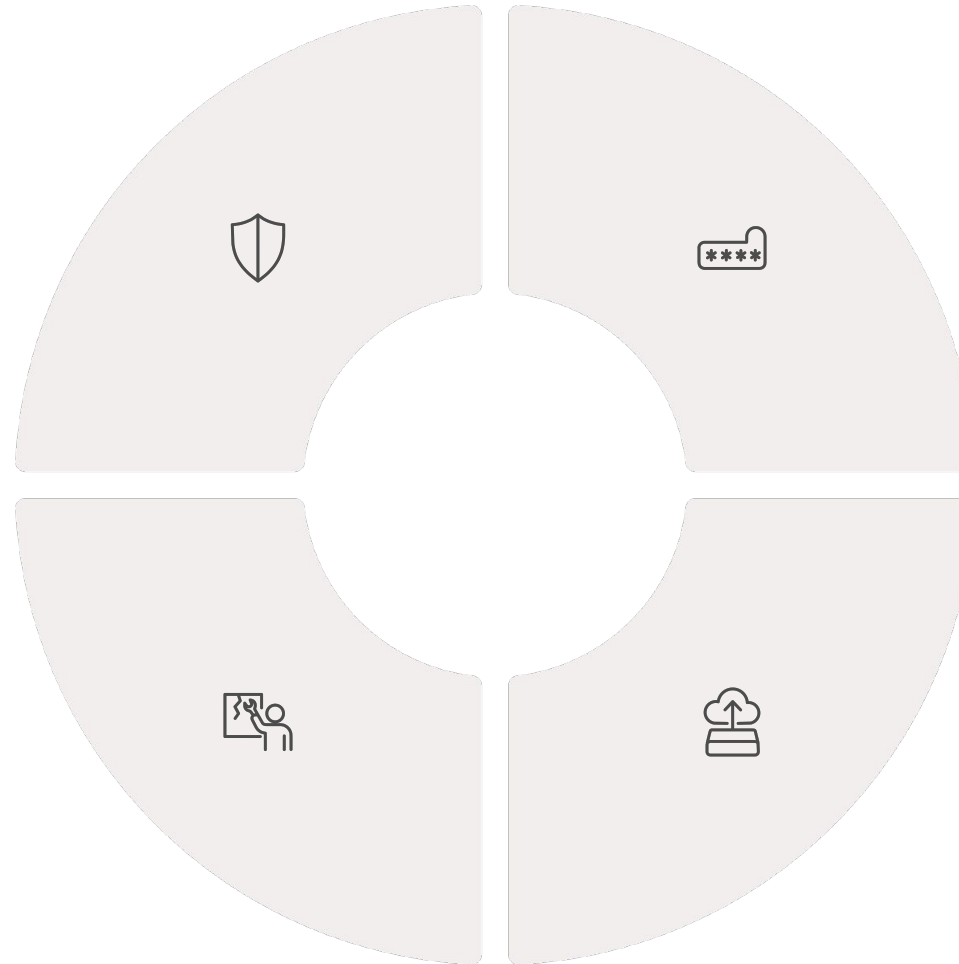
Data Security, Access Control, Backup and Recovery

Access Control

GRANT and REVOKE permissions
ensure users can only access
authorized data

Recovery Procedures

Log-based rollback and rollforward
restore database to consistent state
after failures



Encryption

Data at rest and in transit is protected
from unauthorized viewing

Backup Strategies

Full, differential, and incremental
backups protect against data loss

These mechanisms work together to ensure **confidentiality, integrity, and availability** of your data.

Non-Relational Databases

Traditional relational databases excel at structured data with fixed schemas, but modern applications need more flexibility for unstructured data, horizontal scaling, and high throughput.



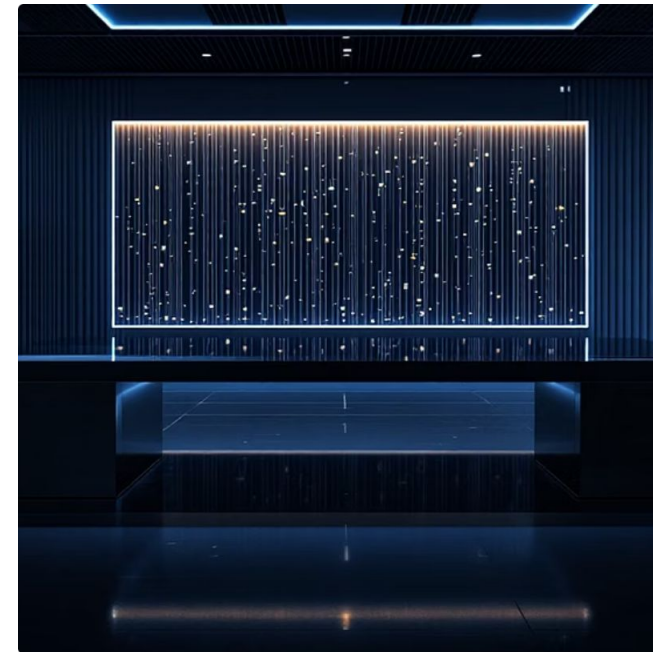
Document Databases

Store data as JSON-like documents (MongoDB, CouchDB)



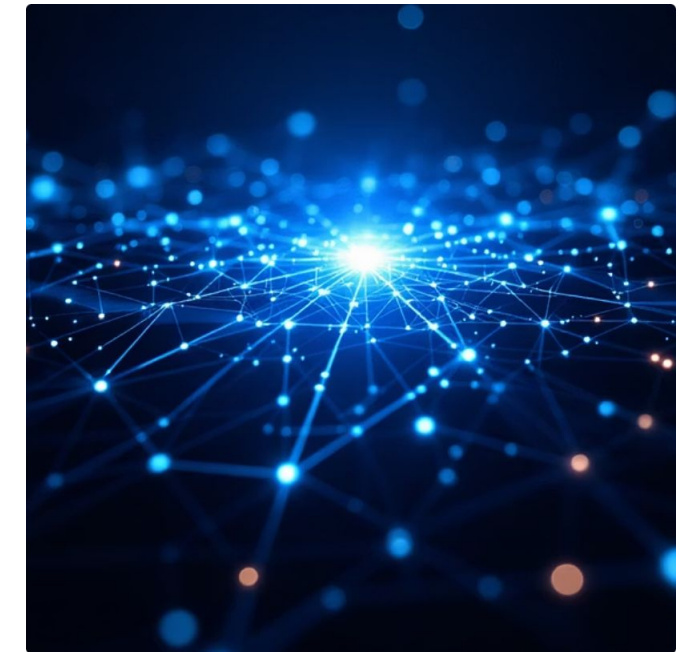
Key-Value Stores

Simple get/set operations for ultra-fast access (Redis, DynamoDB)



Wide-Column Stores

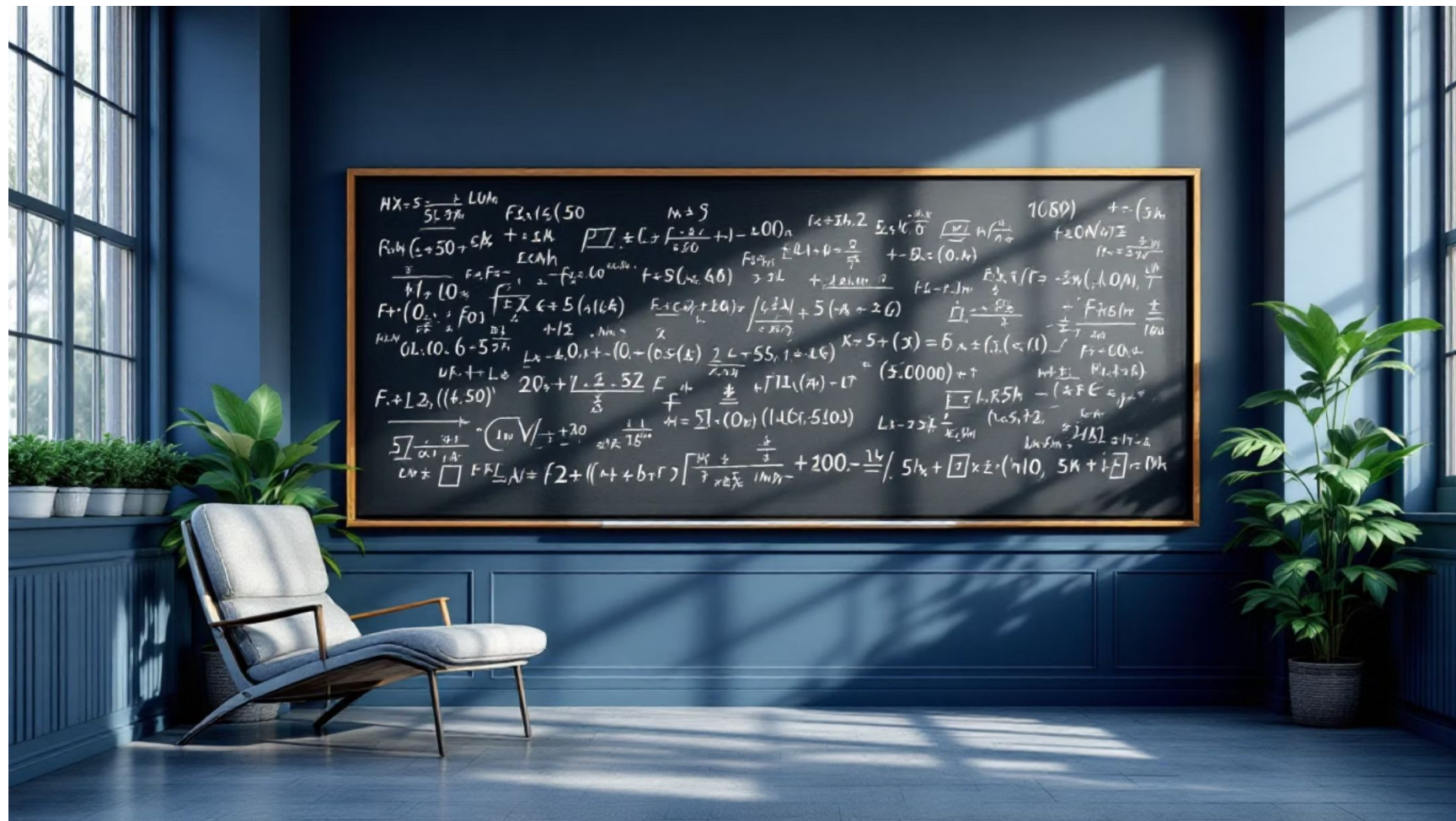
Flexible columns per row for analytical workloads (Cassandra, HBase)



Graph Databases

Store relationships as first-class entities (Neo4j, JanusGraph)

Relational Calculus



The Declarative Query Approach

Relational calculus is a formal query language that lets you describe *what* data you want, not *how* to retrieve it. Unlike relational algebra's procedural operations, calculus uses logical expressions to specify queries. Think of it as asking questions in pure logic: "Find all customers with balance > 1000" rather than step-by-step retrieval instructions.

Thank you!

